COL2010 – Homework 5: Regression

Lectures 1–3 only: least squares, basis functions, overfitting and regularization

11 tasks

What you may use: a Python/NumPy reference

The functions and operations below are useful for implementing the tasks in this homework. You may also use ordinary Python control flow, lists, dictionaries, and tuples. You do not need to search the entire NumPy API.

Library restriction. Use Python's standard library and NumPy for all assessed functions. Do not use `sklearn` regression estimators, automated model-selection routines, or optimization libraries to replace a requested implementation. The supplied setup may use `sklearn.datasets` to obtain data; `matplotlib` may be used for Explorations.

Library/function	Use in this homework
`np.asarray, np.array`	Convert inputs into NumPy arrays; construct coefficient and prediction arrays.
`np.zeros, np.ones, np.eye`	Create intercept columns, initialized arrays and diagonal penalty matrices.
`np.arange, np.linspace`	Polynomial exponents, index sequences and plotting grids.
`np.column_stack, np.stack`	Assemble design matrices and collect results across parameter settings.
`np.sum, np.mean, np.std`	Residual sums of squares, mean squared error, feature standardization and average experiment results.
`np.abs, np.max, np.sqrt`	Orthogonality diagnostics, error magnitudes and root-mean-square error.
`np.floor`	Compute integer sample counts for dataset splits.
`np.power, np.vander, X ** degree`	Construct polynomial basis functions and design matrices.
`np.linalg.lstsq`	Compute least-squares coefficients without explicitly inverting the normal-equation matrix.
`np.linalg.solve`	Solve a nonsingular linear system, including regularized normal equations.
`np.linalg.norm, np.linalg.matrix_rank`	Vector norms and diagnostics for rank-deficient design matrices.
`A @ B, A.T`	Matrix multiplication and transposition; fitted values and normal-equation residuals.
`np.random.default_rng(seed)`	Create a reproducible random-number generator.
`rng.permutation, rng.choice`	Shuffle/split observations and sample training subsets for learning curves.
Boolean indexing and array slicing	Select train/validation/test rows without changing their order unexpectedly.
`np.argmin, np.asarray`	Select the smallest validation error and return numeric score arrays.
`matplotlib.pyplot.plot, scatter`	Visualize fitted curves, residuals, validation errors and learning curves (Explorations only).

Files, restrictions, and submission

Run `python setup.py` in the supplied directory to regenerate the datasets. Python 3.10+ and NumPy are required; matplotlib is for exploration plots. The setup uses scikit-learn only to load its bundled diabetes dataset, not as a student regression solver. Do not use scikit-learn estimators,

automated model selection, or optimization libraries in the assessed functions. Use NumPy linear algebra. Do not modify function signatures or input helpers. Submit your completed `solutions.py` (copied from `skeleton-solutions.py`)

How to obtain inputs and test each function

For every Task N, `inputs.py` defines `get_taskN_inputs()` returning a tuple in the exact positional order of the function signature.

```
from inputs import get_task1_inputs, get_task9_inputs, load_dataset
from solutions import design_matrix, select_degree
Phi = design_matrix(*get_task1_inputs())
best_degree, validation_errors = select_degree(*get_task9_inputs())
X, y, V, v, T, t = load_dataset("quadratic")
# Train on X,y; tune using V,v; evaluate once on T,t.
```

Dataset laboratory. `quadratic`: low-degree polynomial signal; `oscillatory`: a curve difficult for low-degree polynomials; `small_sample`: high variance/overfitting; `outliers`: sensitivity of squared error; `diabetes`: real multivariate regression data bundled with scikit-learn. Use the one-dimensional synthetic datasets for polynomial tasks. The optional `python setup.py --download` downloads California housing for further exploration only; Internet is not needed for assessed tasks. `available_datasets()` lists local datasets. Synthetic data are reproducible.
Conventions. The first basis column is the constant intercept. Mean squared error uses $N^{-1}\sum_n(y_n - \hat{y}_n)^2$. Ridge does not apply penalty on intercept term. Ties in model selection are resolved by the earliest supplied candidate. Train/validation/test separation must be respected.

Part I – Foundations

Task 1. Polynomial design matrix
Description. Construct the polynomial basis matrix from a one-dimensional vector of N inputs. Column j must contain x raised to power j, for j=0,...,degree. The first column must therefore be all ones. Preserve the original input order and do not modify X.
Function signature.

```
def design_matrix(X: np.ndarray, degree: int) -> np.ndarray:
    ...
```

Expected Output. np.ndarray of shape (N, degree+1), dtype compatible with floating-point calculations. For X=[-1,0,2], degree=2, return [[1,-1,1],[1,0,0],[1,2,4]].

How to call.

```
from inputs import get_task1_inputs
from solutions import design_matrix
result = design_matrix(*get_task1_inputs())
print(result)
```

Task 2. Predict using a design matrix
Description. Compute the prediction for every observation using matrix multiplication Phi @ w. Phi has N rows and M columns; w has length M. No fitting takes place in this task.
Function signature.

```
def predict(Phi: np.ndarray, w: np.ndarray) -> np.ndarray:
    ...
```

Expected Output. np.ndarray of shape (N,). For Phi=[[1,0],[1,2]] and w=[3,4], return [3,11].

How to call.

```
from inputs import get_task2_inputs
from solutions import predict
result = predict(*get_task2_inputs())
print(result)
```

Task 3. Mean squared error
Description. Compute the arithmetic mean of squared residuals (target minus prediction). Do not take a square root and do not divide by the number of parameters.
Function signature.

```
def mse(y: np.ndarray, y_pred: np.ndarray) -> float:
    ...
```

Expected Output. float. For y=[1,3] and y_pred=[2,1], return 2.5.

How to call.

```
from inputs import get_task3_inputs
from solutions import mse
result = mse(*get_task3_inputs())
print(result)
```

Exploration. Change one target by a large amount; compare how the squared error changes relative to the original fit.

Task 4. Ordinary least-squares fitting
Description. Find coefficients minimizing the sum of squared residuals for the supplied design matrix. Use NumPy linear algebra, preferably lstsq, rather than explicit matrix inversion. The first column of Phi will be an all 1's columns.
Function signature.

```
def ols(Phi: np.ndarray, y: np.ndarray) -> np.ndarray:
    ...
```

Expected Output. np.ndarray of shape (M,). For Phi=[[1,0],[1,1],[1,2]] and y=[1,3,5], the answer is approximately [1,2].

How to call.

```
from inputs import get_task4_inputs
from solutions import ols
result = ols(*get_task4_inputs())
print(result)
```

Exploration. Create a nearly duplicate feature column. Compare fitted coefficients, training error and predictions; explain instability of coefficients under collinearity.

Task 5. Projection and residual orthogonality

Description. Fit ordinary least squares, compute the fitted target vector and residual vector e=y-Phi@w, then compute the maximum absolute coordinate of Phi.T@e. This value diagnoses the normal-equation orthogonality condition.

Function signature.

```
def projection_diagnostics(Phi: np.ndarray, y: np.ndarray) -> tuple[np.ndarray, np.
    ndarray, float]:
    ...
```

> **Expected Output.** tuple[np.ndarray, np.ndarray, float]: fitted values shape (N,), residuals shape (N,), and maximum orthogonality violation. The violation should be near zero on a well-conditioned full-rank example.

How to call.

```
from inputs import get_task5_inputs
from solutions import projection_diagnostics
result = projection_diagnostics(*get_task5_inputs())
print(result)
```

Task 6. Ridge regression with unpenalized intercept

Description. Minimize $\|y - \Phi w\|_2^2 + \lambda \sum_{j \geq 1} w_j^2$. The first column is the intercept and its coefficient must not be penalized. Solve the resulting linear system stably. Lambda is nonnegative.

Function signature.

```
def ridge(Phi: np.ndarray, y: np.ndarray, lam: float) -> np.ndarray:
    ...
```

> **Expected Output.** np.ndarray of shape (M,). For lam=0 on a full-rank matrix, the result should agree with ordinary least squares up to numerical tolerance.

How to call.

```
from inputs import get_task6_inputs
from solutions import ridge
result = ridge(*get_task6_inputs())
print(result)
```

Exploration. Fit a high-degree polynomial on small_sample at several lambdas. Compare coefficient norms and training/validation error; explain why regularization can improve generalization.

Task 7. Reproducible train/validation/test split

Description. Return a random permutation of row indices split into disjoint training, validation and test sets. Training count is floor(n*train_fraction), validation count is floor(n*val_fraction), and the remainder goes to test. Use np.random.default_rng(seed). Do not split feature and target arrays independently.

Function signature.

```
def split_indices(n: int, train_fraction: float, val_fraction: float, seed: int) ->
    tuple[np.ndarray, np.ndarray, np.ndarray]:
    ...
```

Expected Output. tuple[np.ndarray, np.ndarray, np.ndarray] of one-dimensional integer index arrays, with lengths defined above. Together they must contain each integer 0,...,n-1 exactly once.

How to call.

```
from inputs import get_task7_inputs
from solutions import split_indices
result = split_indices(*get_task7_inputs())
print(result)
```

Exploration. Try multiple seeds and fractions; verify no overlap. Explain why selecting hyperparameters on the test set biases the final estimate.

Task 8. Feature normalization
Description. Given a 2D array of features, standardize each column apart from the first one. Standardization must be $(x - \text{mean})/(\text{std} + \text{eps})$. The first column must remain unchanged. Do not modify the input array.
Function signature.

```
def standardize(X: np.ndarray, eps: float = 1e-8) -> np.ndarray:
    ...
```

Expected Output. np.ndarray of shape (N, M), matching the input shape, with the first column unchanged and all remaining columns standardized. For X=[[1,-1],[1,0],[1,1]], the result is approximately [[1,-1.225],[1,0],[1,1.225]].

How to call.

```
from inputs import get_task8_inputs
from solutions import standardize
result = standardize(*get_task8_inputs())
print(result)
```

Part II – Involved experiments

Task 9. Polynomial degree selection
Description. For each candidate degree in the supplied order, construct polynomial design matrices on training and validation inputs, fit OLS on training only, and compute validation MSE. Select the first degree achieving the smallest validation MSE. Do not use test data.
Function signature.

```
def select_degree(x_train: np.ndarray, y_train: np.ndarray, x_val: np.ndarray, y_val
    : np.ndarray, degrees: tuple[int, ...]) -> tuple[int, np.ndarray]:
    ...
```

Expected Output. tuple[int, np.ndarray]: best degree as a Python int and validation MSEs as a float array of shape (len(degrees),), in exactly the supplied order.

How to call.

```
from inputs import get_task9_inputs
from solutions import select_degree
result = select_degree(*get_task9_inputs())
print(result)
```

Exploration. On quadratic and oscillatory data, plot training/validation MSE against degree and draw the chosen curve. Discuss underfitting and overfitting.

Task 10. Ridge regularization selection

Description. For a fixed polynomial degree, fit ridge models for each supplied nonnegative lambda, in the given order. Use training observations only for fitting and validation observations for choosing lambda. Select the first minimum. Do not standardize validation data independently or inspect test targets.

Function signature.

```
def select_ridge(x_train: np.ndarray, y_train: np.ndarray, x_val: np.ndarray, y_val:
    np.ndarray, degree: int, lambdas: tuple[float, ...]) -> tuple[float, np.ndarray
    ]:
    ...
```

Expected Output. tuple[float, np.ndarray]: selected lambda as a float and validation MSE array of shape (len(lambdas),), preserving input order.

How to call.

```
from inputs import get_task10_inputs
from solutions import select_ridge
result = select_ridge(*get_task10_inputs())
print(result)
```

Exploration. Compare OLS (lambda=0) and regularized high-degree fits on small_sample and outliers. Plot fitted curves, validation error and coefficient norm versus lambda.

Task 11. Learning curves and sample size

Description. Shuffle training rows once using the supplied seed. For each requested training size, take the first size rows of that fixed permutation, fit an OLS polynomial of the given degree, and evaluate MSE on that subset and on the unchanged validation set. Do not resample independently for each size.

Function signature.

```
def learning_curve(x_train: np.ndarray, y_train: np.ndarray, x_val: np.ndarray,
    y_val: np.ndarray, degree: int, sizes: tuple[int, ...], seed: int) -> tuple[np.
    ndarray, np.ndarray]:
    ...
```

Expected Output. tuple[np.ndarray, np.ndarray]: training MSE array and validation MSE array, each shape (len(sizes),), in the given size order.

How to call.

```
from inputs import get_task11_inputs
from solutions import learning_curve
result = learning_curve(*get_task11_inputs())
print(result)
```

Exploration. Plot both curves for degrees 1, 3 and 9. Explain how the train-validation gap changes as the sample size grows and why a flexible model can overfit small samples.